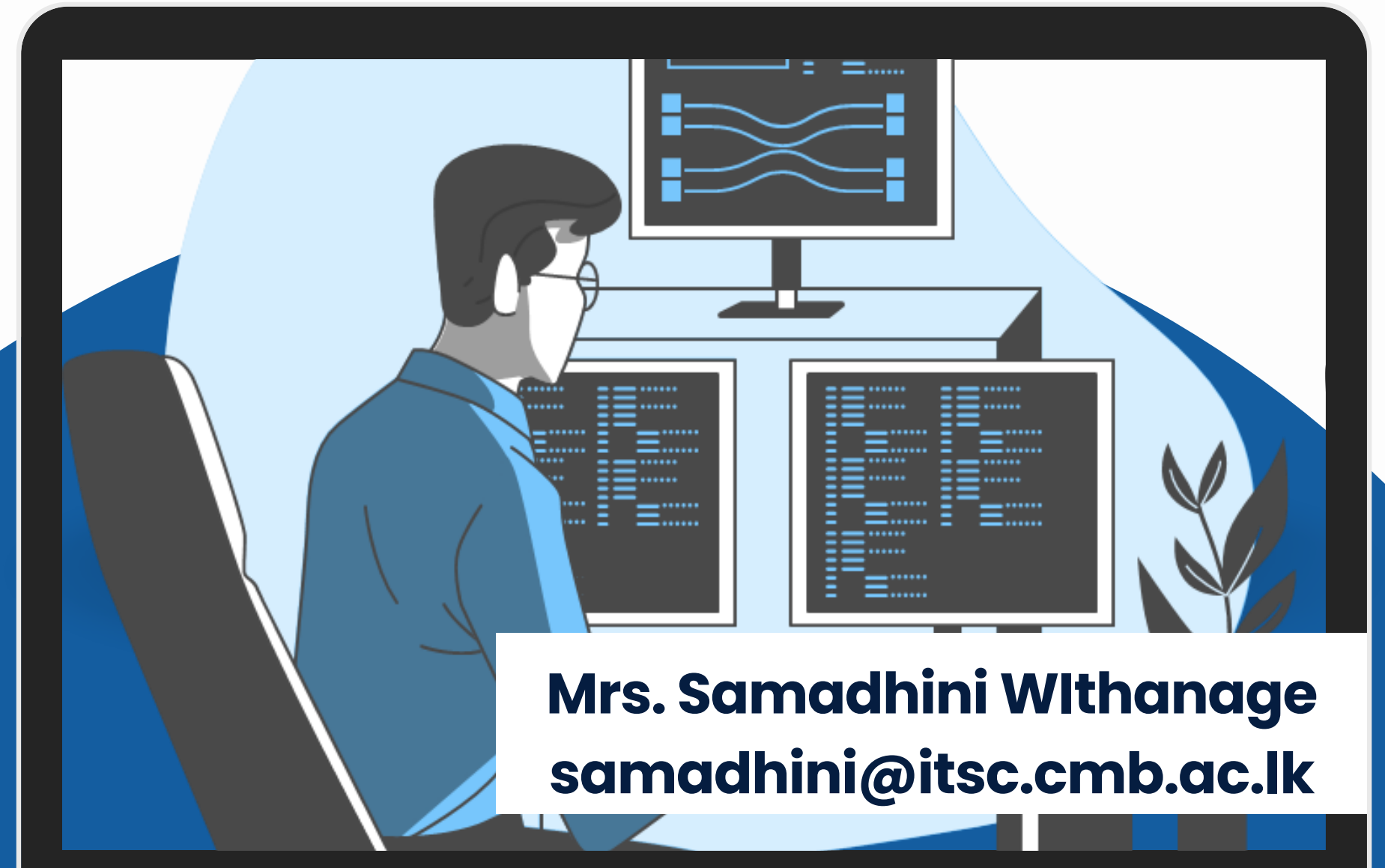


IT 3003

Advanced Programming Techniques



Mrs. Samadhini Withanage
samadhini@itsc.cmb.ac.lk

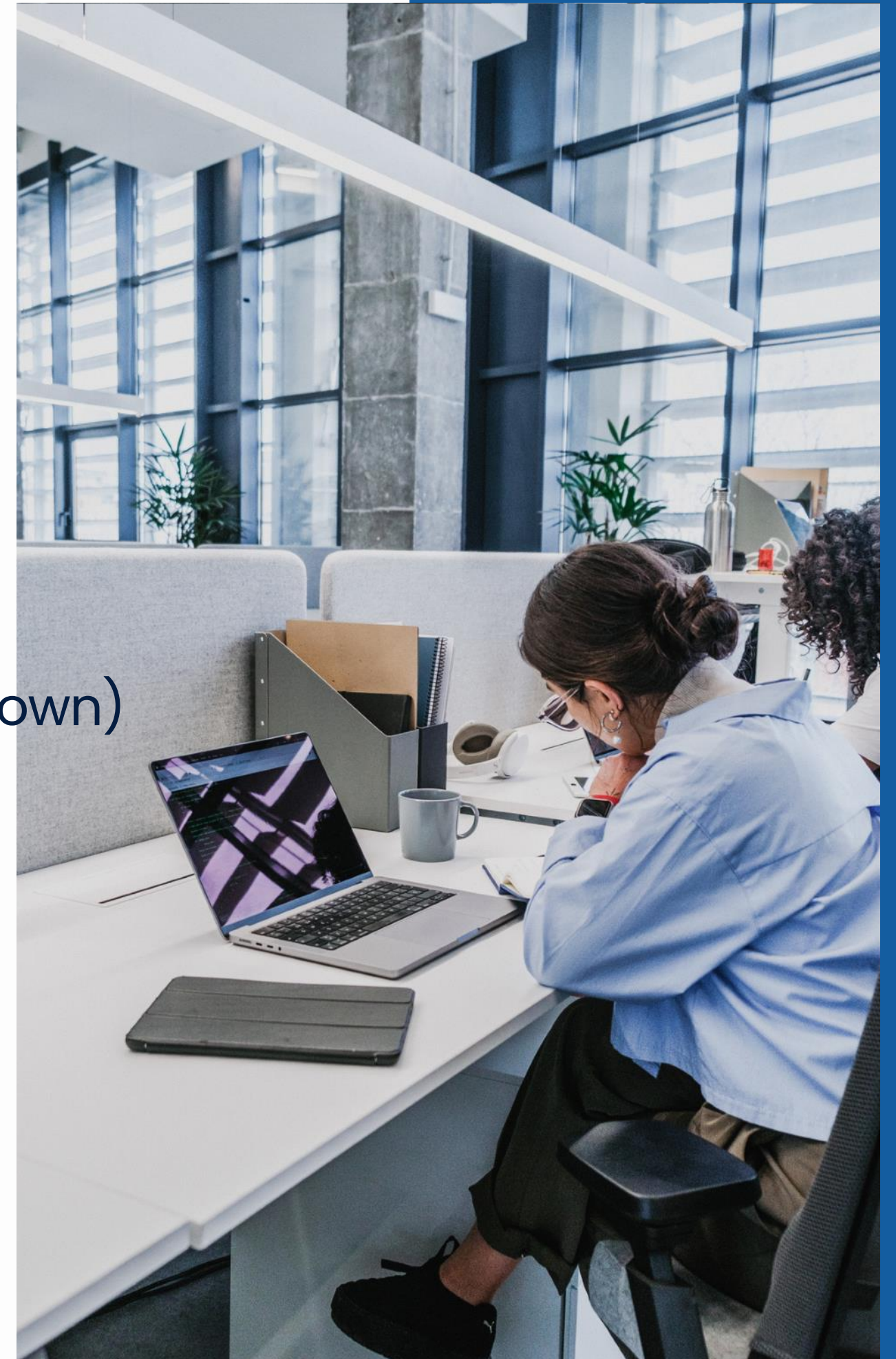
Lesson 1

Java Basic



Lesson Outline

- ▶ Introduction to Java
- ▶ Features & Characteristics
- ▶ Java Environment (JDK, JRE, JVM)
- ▶ Requirements for Java Development
- ▶ Java Program Structure (main method breakdown)
- ▶ Command line compilation & execution
- ▶ Comments & Documentation (JavaDoc intro)
- ▶ Basic Syntax
- ▶ Data Types (Primitive & Reference)
- ▶ Variables (Local, Instance, Static)
- ▶ Arrays
- ▶ Methods



Introduction to Java

- ▶ Java is a **high-level, object-oriented programming language**
- ▶ Developed by **Sun Microsystems (1995)** (now owned by Oracle)
- ▶ Designed to be **platform-independent** (like Windows, Linux, or macOS)
- ▶ Used for:
 - Web applications
 - Mobile apps (Android)
 - Desktop software

Features & Characteristics



Object-Oriented (OOP)

Java is based on objects and classes, which helps organize code and makes it reusable and easier to manage.



Platform Independent

Java programs run on any system with a **JVM** — *write once, run anywhere*.



Secure & Robust

Java has built-in security features and strong error handling, making programs safer and more reliable.

Features & Characteristics Cont..



Multithreaded

Java can run multiple tasks at the same time, improving efficiency and performance.



High Performance (with JVM optimization)

Although not as fast as some low-level languages, Java is optimized by the JVM to run efficiently.



Automatic Memory Management (Garbage Collection)

Java automatically removes unused objects from memory, so developers don't need to manage memory manually.

Java Environment (JDK, JRE, JVM)

- ▶ **JDK (Java Development Kit)**
Used to develop Java programs
- ▶ **JRE (Java Runtime Environment)**
Provides libraries + JVM to run programs
- ▶ **JVM (Java Virtual Machine)**
Executes Java bytecode

Requirements for Java Development

- ▶ A compatible PC with **JDK (Java Development Kit)** installed
- ▶ A Text Editor – OR – An IDE (Integrated Development Environment)

Text Editor

E.g., Notepad

IDE

E.g. Eclipse IDE,
NetBeans IDE

- ▶ **Java Runtime Environment (JRE)** (to run Java programs)
- ▶ **Environment Variables Setup** (e.g., PATH configuration)
- ▶ Basic Knowledge of Programming Concepts
- ▶ **Internet Access** (for documentation and libraries)
- ▶ References

Java Program Structure

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}
```



Breakdown:

public class → Class definition

main() → Entry point of program

System.out.println() → Output statement

Main Method Breakdown

```
public static void main(String[] args)
```



public → Accessible everywhere

static → No object needed

void → No return value

main → Method name

String[] args → Command-line arguments

Command Line Compilation & Execution

Steps:

▶ Write program → Hello.java

▶ Compile: `javac Hello.java`

▶ Run: `java Hello`

▶ Output: `Hello, Java!`

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}
```

Comments & Documentation

► Types of Comments:

```
// Single-line comment
```

```
/* Multi-line  
   comment */
```

```
/**  
 * JavaDoc comment  
 */
```

JavaDoc is used to generate documentation

Basic Syntax Rules

- ▶ Java is **case-sensitive**
- ▶ Every statement ends with ;
- ▶ Code must be inside a class
- ▶ Execution starts from main()

Data Types



Primitive Types :

These are **basic built-in data types** that store **actual values directly** in memory.

- Int → whole numbers

`10` , `-5` , `1000`

- Float → decimal numbers (less precision)

`2.5f` , `-1.2f` , `0.0f`

- Double → decimal numbers (more precision)

`3.14` , `-0.75` , `100.123`

Data Types Cont....

- char → single character

```
3.14, -0.75, 100.123
```

- Boolean → true or false

```
true, false
```

- Byte → very small integer (-128 to 127)

```
10, -50, 127, -128
```

- Short → small integer

```
100, -200, 32767
```

- Long → large integer

```
123456789L, -9000000L
```

Data Types Cont....

```
float num1 = 2.5;    ???
```

Data Types Cont....

Java Data Types & Memory Allocation

Data Type	Size (Memory)	Range / Description
byte	1 byte (8 bits)	-128 to 127
short	2 bytes (16 bits)	-32,768 to 32,767
int	4 bytes (32 bits)	-2^{31} to $2^{31}-1$
long	8 bytes (64 bits)	Very large integers
float	4 bytes (32 bits)	Decimal (less precision)
double	8 bytes (64 bits)	Decimal (high precision)
char	2 bytes (16 bits)	Single character (Unicode)
boolean	~1 bit*	true / false

Data Types Cont....

▶ Reference Types

These store **references (addresses)** to objects, not the actual data.
(Reference types store the memory location of data, not the data itself.)

01. Arrays → Store multiple values of the **same type**

```
int[] numbers = {1, 2, 3};
```


Data Types Cont....

02. Objects → Created from classes
Represent real-world entities

```
Student s = new Student();
```

03. Strings → Used to store text In Java,
Strings are objects

```
String name = "John";
```

Variables



A **variable** is a named memory location used to **store data** in a program.

It holds a value that can change during program execution.

Variable Declaration

Declaration means telling Java:

- the data type
- the variable name

You are creating a variable, but not giving a value yet

```
int num;
```


Variables Cont...

Variable Initialization

Initialization means assigning a value to the variable.

```
num = 10;
```

Declaration + Initialization together

You can also do both in one line

```
int num = 10;
```

Variables Cont...

Types of Variables in Java

1. Local Variables
2. Instance Variables
3. Static Variables

1. Local Variables

- Declared inside a method or block
- Accessible only within that method
- Must be initialized before use

```
public class Test {  
    public static void main(String[] args) {  
        int x = 10;    // local variable  
        System.out.println(x);  
    }  
}
```

Variables Cont...

2. Instance Variables

- Declared inside a class but outside methods
- Belong to an object (instance)
- Each object has its own copy

age is different for each object

```
public class Student {  
    int age;    // instance variable  
  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.age = 20;  
  
        Student s2 = new Student();  
        s2.age = 25;  
  
        System.out.println(s1.age);  
        System.out.println(s2.age);  
    }  
}
```

Variables Cont...

3. Static Variables

Declared using static keyword
Belong to the class, not objects
Shared among all objects

Both objects share the same college

```
public class Student {  
    static String college = "ABC University"; // static variable  
  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        Student s2 = new Student();  
  
        System.out.println(s1.college);  
        System.out.println(s2.college);  
    }  
}
```

Arrays



- An **array** is used to store **multiple values of the same type** in a single variable.
- Instead of creating many variables, you use one array.
- Fixed size

Array Declaration

```
int[] numbers = new int[5];
```



Data Type



Name



Size

Arrays Cont....

Assign values to each position using index

```
numbers[0] = 10;  
numbers[1] = 20;  
numbers[2] = 30;  
numbers[3] = 40;  
numbers[4] = 50;
```

```
numbers[0] = 10;
```

↑
Array
Name

↑
Index No

↑
Value

Arrays Cont....

Printing Values

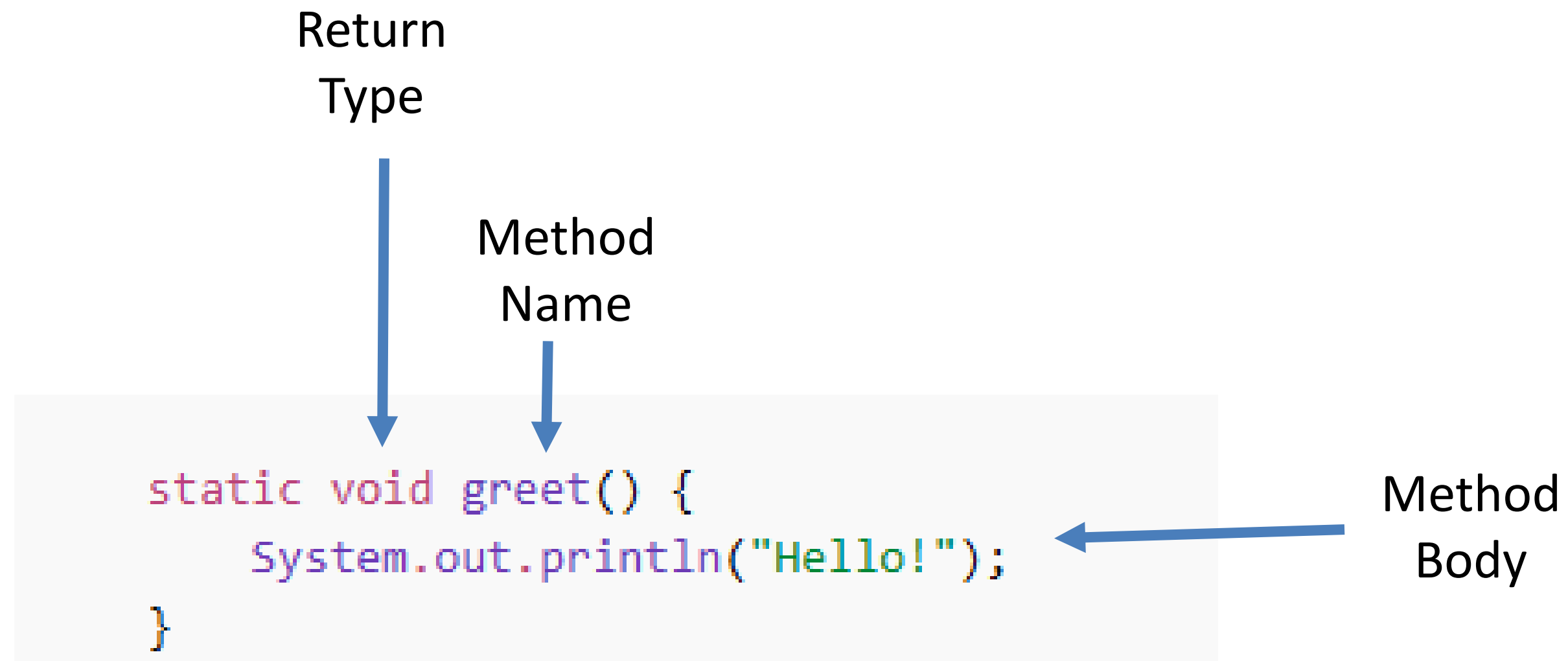
```
System.out.println(numbers[0]);  
System.out.println(numbers[1]);  
System.out.println(numbers[2]);  
System.out.println(numbers[3]);  
System.out.println(numbers[4]);
```

Arrays Cont....

```
int[] numbers = {10, 20, 30, 40};
```


Methods

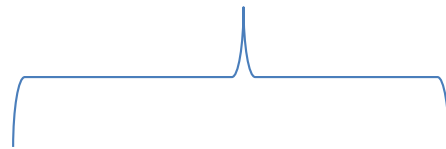
- ▶ A method is a block of code that performs a task Improves code reusability



Methods Cont...

Method with Parameters

Parameters

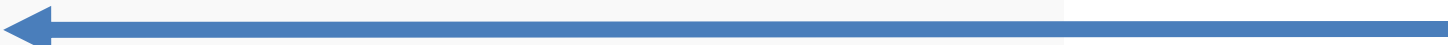


```
static void add(int a, int b) {  
    System.out.println(a + b);  
}
```

Methods Cont...

```
public class MethodExample {  
  
    static void add(int a, int b) {  
        System.out.println(a + b);  
    }  
  
    public static void main(String[] args) {  
        add(5, 3);  
    }  
}
```

Method
Call



Take-Home Exercise

Question 1

Explain what JavaDoc is and describe how it is used to document a Java program. Include examples of at least three JavaDoc tags and explain their purpose.

Question 2

Describe the process of installing Java (JDK) on your operating system (Windows, macOS, or Linux). Include the steps for setting environment variables and explain how to verify that Java has been installed correctly.

The background is a blurred office scene with people working at desks, overlaid with a semi-transparent blue filter. A white rectangular box is centered on the image, containing the text "ANY QUESTIONS ?" in a bold, white, sans-serif font. The word "ANY" is on the top line, and "QUESTIONS ?" is on the bottom line.

**ANY
QUESTIONS ?**



THANK YOU!

